

CS3006 Assignment 2

Performance Analysis of Parallel and SIMD Programs on a Dual Core Laptop

CS3006, Parallel and Distributed Computing

Name: Fatima Tahir

Roll Number: 23L-0799

Instructor: Dr. Abdul Qadeer

FAST School of Computing, National University of Computer and Emerging Sciences
Lahore, Pakistan

Part 0: Machine Declaration

All measurements in this report were taken on my own laptop, a Dell Latitude 3490, following the addendum's instructions for students without access to the Stanford myth machines. The relevant hardware and software details, gathered with `lscpu`, `gcc --version`, and `uname -a`, are listed below.

Quantity	Value
Machine	Dell Latitude 3490
CPU model	Intel Core i5-7200U (7th generation)
Physical cores	2
Threads per core	2 (Hyper Threading)
Logical CPUs	4
Max clock speed	3100 MHz (base 2.50 GHz)
SIMD support	AVX2, 8 wide single precision (256 bit)
L1d cache	64 KiB (2 instances)
L2 cache	512 KiB (2 instances)
L3 cache	3 MiB (1 instance)
RAM	8019348 kB (about 7.6 GiB usable)
OS	Ubuntu 24.04, dual booted alongside Windows
Kernel	Linux fatima-Latitude-3490 7.0.0-31-generic, SMP PREEMPT_DYNAMIC
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
ISPC version	1.31.0

Because Plog works out to $P_{\text{phys}} \times t$, 2 times 2 equals 4, this machine has only 2 physical cores even though it exposes 4 logical CPUs to the operating system. The addendum accounts for this directly: every performance target in the sections below is quoted against a 2 core, 4 thread ceiling rather than the 4 core, 8 thread myth machines the original Stanford handout assumes. The two logical CPUs beyond the physical core count are Hyper Threading siblings, so they share execution units with their physical core instead of behaving like independent cores. I confirmed this using `lscpu`, which reports 2 cores per socket and 2 threads per core, and `nproc`, which reports 4.

Build Notes

The starter code needed two small changes before it would compile under GCC 13. Program 1's `main.cpp` was missing an include for `cstring`, and `mandelbrotThread.cpp` was missing an include for `cstdlib`. Both are one line additions with no effect on program behaviour, only on whether the code compiles at all with a modern compiler. Everything else built and ran out of the box.

A note on repository history. Partway through this assignment my working directory was deleted from disk, for reasons I still cannot pin down (I checked bash history for stray `rm` commands and checked `lost` and `found`, neither turned up anything). Programs 1 through 4 had already been completed and committed at that point, but none of it had been pushed to GitHub yet, so it was lost along with the rest

of the repository. I redid Programs 1 through 4 in a fresh repository and pushed after every single commit from that point on, which is the workflow reflected in the final git history submitted with this assignment. I am mentioning this here in the interest of being upfront about it rather than because the addendum asks for it.

Program 1: Parallel Fractal Generation Using Threads

The starter code spawns a thread that does nothing but print a message. My job was to fill in `workerThreadStart` so each thread actually computes part of the image, then improve on the naive version until speedup gets close to what this machine can deliver.

Basic decomposition

The first version splits the image into equal horizontal strips, one per thread, computed by dividing the image height by the thread count and giving each thread a contiguous block of rows starting at `threadId` times that block size. This is the most direct reading of the assignment and it gave a 2 thread speedup of about 1.97x to 1.99x across repeated runs, close to ideal for 2 threads.

Why speedup does not stay linear

Once the thread count goes past 2, equal sized row blocks stop being equal amounts of work. The Mandelbrot set is not evenly expensive to compute. Pixels far outside the set escape after only a few iterations, while pixels near the boundary of the set run close to the full iteration limit before the loop gives up. Because the set itself sits roughly in the middle of the image for View 1, a block decomposition tends to hand one or two threads a strip that is mostly boundary, while the rest of the threads get strips that are mostly empty background. Whichever thread lands on the expensive strip becomes the bottleneck, and the other threads finish early and simply wait at the join. This is visible directly in the measured data: 3 threads performed worse than 2 threads in early testing (a smaller thread count that happens to align better with the image's row distribution can outperform a larger one that splits the expensive region unevenly), which only makes sense if the work is not being split evenly.

The fix does not need any locking or communication between threads. Instead of giving each thread a contiguous block, I interleaved rows: thread `t` computes row `t`, then row `t` plus `numThreads`, then row `t` plus 2 times `numThreads`, and so on until it runs out of rows. Every thread now gets a mix of rows from all over the image, so whatever expensive region exists gets spread across all threads roughly evenly instead of landing on one of them. This also fixes a correctness bug in the block version, where thread counts that do not divide the image height evenly (7, for a height of 720) leave a handful of rows uncomputed because of integer division truncation. Interleaving does not have this problem since every row is claimed by exactly one thread regardless of how the row count and thread count relate.

Measured results

The table below is the final 5 repetition measurement pass on View 1, run on a quiet machine with nothing else active.

Threads	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max
4	2.93x	3.39x	3.39x	3.39x	3.44x	2.93x	3.44x
8	2.97x	3.30x	3.48x	3.53x	3.44x	2.97x	3.53x

A single run at each of 2, 3, 4, 6, 7, and 8 threads was also recorded earlier for View 1, giving 2.21x, 2.97x, 3.29x, 3.36x, 3.33x, and 3.45x respectively, which matches the shape of the 5 repetition data above. In both tables the first run in a batch tends to come in a little lower than the rest, which I take to be a cold cache or thermal warm up effect rather than anything about the decomposition itself, since the remaining four runs at each thread count cluster tightly together.

Speedup plateaus around 3.3x to 3.4x from 4 threads onward and does not meaningfully improve at 7 or 8 threads. This matches the addendum's own adjusted target for a 2 core, 4 thread machine of roughly 3x to 3.5x, well below the Stanford handout's 7x to 8x target, which assumes 4 physical cores. Once thread count exceeds the 4 available hardware threads, additional threads are just being time sliced onto the same 4 execution contexts, so there is no more real parallelism left to extract, only extra scheduling overhead. This is also why 16 threads did not measurably outperform 8 threads when I checked it during development. Programs 1 and 3 both hit this exact ceiling for the same underlying reason, that the hardware only has 4 threads to give, regardless of how many software threads the program launches.

Program 2: Vectorizing Code Using SIMD Intrinsics

This program asks for a vectorized implementation of `clampedExp` using the course's fake vector intrinsics, which behave like real SIMD instructions but are simulated in software so mistakes are easy to catch and debug.

Implementation approach

The serial version treats an exponent of 0 as a special case and otherwise multiplies in a plain loop. I rewrote the logic so every lane starts at a result of 1.0 and is multiplied by its own value exactly as many times as its own exponent, using a per lane count that gets decremented and masked off once it reaches zero. This removes the need for a separate zero exponent branch, since a lane with exponent 0 simply never enters the multiply loop. A mask built from `init_ones` handles the case where `N` is not a multiple of `VECTOR_WIDTH`, since only the first `N` minus `i` lanes should be marked active on the last, partial group. After the count loop finishes, a second mask catches any lane whose result exceeds 9.999999 and clamps it in place.

The implementation passed correctness checking on the first attempt that produced a result: 'Results matched with answer' and 'Passed' for every value of `N` tried, including `N` equal to 3, which the handout specifically calls out as a case worth testing since it does not divide `VECTOR_WIDTH` evenly.

Vector width sweep

The handout asks for utilization at `N` equal to 10000 across vector widths of 2, 4, 8, and 16, obtained by changing the `VECTOR_WIDTH` define in `CS149intrin.h` and rebuilding before each run.

Vector width	Utilization
2	88.6%
4	83.6%
8	81.0%
16	79.7%

Utilization goes down as vector width goes up. The reason comes from how the inner multiply loop is written: a group of lanes cannot move on to the next group until every lane inside it has finished counting down its own exponent, so a lane with a small exponent sits idle, fully masked off, for however many extra iterations the slowest lane in its group still needs. Exponents in this program are drawn at random between 0 and 9, so the more lanes are grouped together, the more likely it becomes that at least one of them draws a large exponent and drags the whole group out. A wider vector simply has more lanes exposed to this effect at once, which is why utilization falls steadily from 88.6% at width 2 down to 79.7% at width 16. Five repetitions at the default width of 4 produced the identical utilization figure of 83.6% every time, since the exponents come from rand called without an explicit seed and so produce the same pseudo random sequence on every fresh run of the program.

I left the array sum extra credit unattempted given the time available for this assignment, so arraySumVector still reports 0 and fails its check. VECTOR_WIDTH was reset to its default value of 4 before final submission.

Program 3: Parallel Fractal Generation Using ISPC

Part 1: SIMD only

Running mandelbrot_ispc as provided, with no code changes needed, gave a speedup over the serial version in the range of 4.85x to 5.06x across five repetitions, well under the 8x ceiling that an 8 wide AVX2 target should in principle allow.

Configuration	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max
ISPC, no tasks	5.03x	4.85x	5.06x	5.06x	5.04x	4.85x	5.06x

The shortfall comes from the same load imbalance discussed in Program 1, just moved one level down. ISPC groups nearby pixels into a gang of 8 lanes that all execute in lockstep, so a gang only finishes once its slowest lane finishes. Because the Mandelbrot computation's cost varies enormously between pixels that escape almost immediately and pixels near the set boundary that iterate all the way to the limit, any gang that straddles the boundary is stuck waiting on its slowest lane while the fast lanes in the same gang sit idle, masked off, doing nothing. This is a SIMD lane version of the exact imbalance mechanism that hurt the naive block decomposition in Program 1.

Part 2: ISPC tasks

The starter code's mandelbrot_ispc_withtasks launches only 2 tasks, giving a task based speedup of about 8.50x in a baseline run, worse than it should be for a 2 core machine. I changed both the task count and the corresponding rowsPerTask computation from 2 to 32. Thirty two was chosen for two reasons. It divides the image height of 800 evenly, avoiding a correctness bug where a task count that does not divide the height leaves the last few rows uncomputed (I hit exactly this failure mode when I first tried 64 tasks, since 800 divided by 64 truncates to 12 and leaves the final 32 rows blank). It also gives the task scheduler enough small units of work to spread evenly across the machine's 2 cores, which smooths out the same per pixel load imbalance described above rather than concentrating it inside just 2 large tasks.

Configuration	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max
---------------	-------	-------	-------	-------	-------	-----	-----

ISPC with tasks, launch[32]	16.17x	14.12x	16.46x	16.40x	16.62x	14.12x	16.62x
-----------------------------	--------	--------	--------	--------	--------	--------	--------

This machine's addendum adjusted ceiling for Program 3 Part 2 is the number of physical cores times the SIMD width, 2 times 8, which equals 16x, well below the Stanford handout's 32x figure that assumes 4 physical cores. The measured results cluster close to that 16x ceiling, with one lower outlier at 14.12x that I attribute to ordinary OS scheduling noise given how short the absolute runtimes involved are, under 20 milliseconds per run.

Program 4: Iterative sqrt

This program needed no changes to the ISPC or C++ logic. The task is entirely about choosing the contents of the input array to demonstrate how much SIMD speedup depends on the data, not just the code.

Best case input

Setting every value in the array to 2.999, the same value repeated for all 20 million entries, gave the best speedup observed for this program.

Configuration	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max
ISPC, no tasks	6.43x	6.47x	6.44x	6.40x	6.48x	6.40x	6.48x
ISPC, with tasks	20.86x	20.93x	20.73x	21.05x	20.84x	20.73x	21.05x

A value of 2.999 sits far from the initial guess of 1.0, so Newton's method needs many iterations to converge, and since every lane holds the identical value, every lane needs exactly the same number of iterations. No lane ever finishes early and sits waiting on another, so there is no divergence penalty at all within a gang. I tried setting every value to exactly 1.0 first, expecting this to also give strong speedup since it is likewise uniform, but that actually performed worse than the random baseline (1.52x versus a baseline of 4.44x), because a value equal to the initial guess converges in essentially one iteration and leaves almost no real computation for the parallelism to speed up. Fixed per element overhead then dominates the tiny amount of actual work, which shrinks the relative speedup even though the absolute runtime is much shorter. The lesson from that failed attempt is what led to choosing 2.999 instead of 1.0 for the uniform value, since the input needs to be both uniform and far from the initial guess to get the best result.

Worst case input

Alternating between 1.0 and 2.999 on every other array element gave the worst measured speedup.

Configuration	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max
ISPC, no tasks	3.30x	3.31x	3.27x	3.30x	3.30x	3.27x	3.31x
ISPC, with tasks	10.86x	10.74x	10.73x	10.60x	10.88x	10.60x	10.88x

Since ISPC's AVX2 target processes 8 lanes per gang, alternating values every single element guarantees that every gang contains both fast converging and slow converging values. The fast lanes finish in about

one iteration and then sit fully idle for every remaining iteration the slow lanes in the same gang still need, which is the largest possible amount of wasted lane cycles that input choice alone can produce. The resulting speedup, 3.27x to 3.31x without tasks, comes in clearly below the random baseline of 4.44x measured with the unmodified starter code, confirming that it is specifically the divergence between neighbouring values, not just the raw amount of computation, that limits SIMD speedup in this workload.

The AVX2 intrinsics extra credit was left unattempted given the time available.

Program 5: BLAS saxpy

saxpy computes result equals scale times X plus Y across 20 million elements. No code changes were required for the required part of this program, only running it and explaining the result.

Metric	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max
Speedup from tasks	0.99x	0.97x	0.98x	0.98x	1.01x	0.97x	1.01x

Task based parallelism gives essentially no benefit here, and in most of the five repeated runs it was very slightly slower than the single core ISPC version rather than faster. The explanation is that saxpy performs only one multiply and one add per element, but has to load two floats and write one float back to memory for every single operation, which makes it memory bandwidth bound rather than compute bound. The single core SIMD version already saturates this machine's shared memory bus on its own, so handing part of the array to additional cores does not unlock any extra throughput, since all cores are drawing from the same limited pipe to RAM. What it does add is the overhead of spawning and synchronizing tasks, with no compensating gain, which is consistent with speedup sitting at or just under 1.0x across every repetition. Given this, I do not think saxpy can be substantially improved by restructuring the code. The operation itself is already the minimum possible amount of arithmetic per byte of memory traffic, so there is no way to reduce the memory pressure further while still computing the same result, and near linear speedup is not achievable for a workload whose bottleneck is bandwidth rather than compute.

Program 6: Making K-Means Faster

The starter code is correct but slow, and the assignment specifically wants the hotspot identified through measurement rather than guessed at. I added timing calls around each of the three functions called every iteration of the main loop, computeAssignments, computeCentroids, and computeCost, using CycleTimer, and ran the unmodified program on a small test dataset (100000 points, 100 dimensions, 3 clusters) to see where the time was actually going.

Profiling

Iteration	computeAssignments	computeCentroids	computeCost
0	49.04 ms	7.26 ms	13.81 ms
10	55.08 ms	9.18 ms	15.09 ms
40	49.47 ms	7.25 ms	13.87 ms
77	50.03 ms	8.31 ms	14.10 ms

computeAssignments consistently takes roughly three and a half times longer than computeCentroids and computeCost combined, and this held true across every iteration I checked, not just an unlucky sample. This matches what the algorithm should do on paper. computeAssignments has to compare every one of the M data points against every one of the K centroids, so its cost scales with K times M times N, while computeCentroids and computeCost each only need one pass over the M points, so their cost scales with M times N. With K equal to 3, computeAssignments is doing roughly three times the raw work of the other two functions by construction, which lines up closely with the observed timing gap. This made computeAssignments the clear target to parallelize, and the assignment's rule that only one of dist, computeAssignments, computeCentroids, or computeCost may be parallelized meant I did not need to consider the others further.

Parallelization

computeAssignments is parallelized by splitting the M data points across 4 threads rather than splitting the K centroids, since K is only 3 on this dataset and would leave at most 3 way parallelism, well short of what this machine's 4 hardware threads can use. Each thread receives a contiguous range of data point indices and runs an unchanged version of the original per point nearest centroid search over just that range, writing results directly into the shared clusterAssignments array at its own indices, which is safe without any locking since no two threads ever write to the same index. The main thread does its own share of the range directly rather than spawning a fourth `std::thread` for it, which avoids one unnecessary thread creation per iteration. Only `kmeansThread.cpp` was modified, `stoppingConditionMet` was left untouched, and the public interface to `kMeansThread` was not changed, all as required by the assignment.

Full scale measurement

Before final measurement, I regenerated the dataset at full scale (1,000,000 points, 100 dimensions, 3 clusters) using `generate_data.py` and confirmed its checksum matched the expected value of `3a25f24193f4fdca82ee4cb2737fd5bb` at a file size of 804002420 bytes, as specified in the addendum.

Version	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max
Serial (ms)	68124.8	67175.2	67846.7	67375.1	68360.7	67175.2	68360.7
Parallel (ms)	37358.2	34514.8	35420.1	31028.0	29928.3	29928.3	37358.2

Taking the minimum of each set of five runs as the representative time, the achieved speedup is 67175.2 divided by 29928.3, which comes to **2.245x**.

Comparing against an Amdahl ceiling

The addendum's real target for this program is not a flat multiplier but a formula relative to the measured hotspot fraction and this machine's hardware thread count. From the profiling data, computeAssignments accounted for about 459.2 ms of a roughly 665.0 ms total iteration time in the unmodified baseline, giving a measured hotspot fraction f of 0.691. With T equal to 4 hardware threads, Amdahl's Law gives a ceiling of 1 divided by the quantity 1 minus f plus f divided by T , which works out to 1 divided by 0.482, or about 2.08x. The addendum's required threshold is 80% of that ceiling, or about 1.66x. The achieved speedup of 2.245x comfortably clears this threshold, and in fact exceeds even

the Stanford handout's original flat 2.1x target outright, despite this being a 2 core rather than 4 core machine.

As a secondary check, I also computed the Karp Flatt experimentally derived serial fraction from the achieved speedup itself, which came out to about 0.260, noticeably lower than the measured 1 minus f value of 0.309 from the profiling pass. My best explanation is that this reflects ordinary session to session variation, background load or thermal state differing slightly between when the profiling numbers were captured and when the final baseline and parallel timings were captured, rather than the parallel version somehow finding extra parallelism beyond what `computeAssignments` alone provides.

Correctness

Running `plot.py` against the log files produced by the full scale run generated `start.png` and `end.png` showing the same overall clustering pattern as the unmodified starter code (a small number of clearly separated clusters with the centroids visibly moving toward the center of their assigned points between the start and end of the algorithm), confirming that the parallel version still converges to the same result as the original.

Extra Credit

No extra credit items were attempted in this submission given the time available.

Honesty Statement

I confirm that every timing measurement in this report was produced by actually running the corresponding program on the machine described in Part 0, with no fabricated or manually edited values. Where a single run stood out as unusually low, most visibly the first run in several five repetition batches, I have reported it alongside the rest of the batch rather than excluding it, and noted the likely cause (cold cache or thermal warm up) in the surrounding text instead of removing it from the data.